FILE: C:\HCL_TRAINING\server\src\services\resumeParser\textExtractor.js

LINES: 316 | SIZE: 10.0 KB | LANGUAGE: JS

MODULE: PDF, DOCX, TXT Parsing & SHA-256 Deduplication Fingerprinting

```
const fs = require('fs');
const path = require('path');
const crypto = require('crypto');
const pdfParse = require('pdf-parse');
const mammoth = require('mammoth');

/**
 * Calculates SHA-256 hash of a file buffer
 */
function calculateFileHash(buffer) {
  return crypto.createHash('sha256').update(buffer).digest('hex');
}

/**
 * Extracts plain text from PDF, DOCX, or text file
 */
async function extractRawText(filePath, mimeType) {
  const buffer = fs.readFileSync(filePath);
  const ext = path.extname(filePath).toLowerCase();

  let text = '';

  if (ext === '.pdf' || mimeType === 'application/pdf') {
    try {
      const data = await pdfParse(buffer);
      text = data.text || '';
      if (!text || text.trim().length < 40) {
        throw new Error('PDF appears to be scanned or contains only images without selec
> table text layer.');
      }
    } catch (err) {
      if (err.message.includes('scanned')) throw err;
      throw new Error(`Failed to parse PDF document: ${err.message}`);
    }
  } else if (ext === '.docx' || mimeType.includes('wordprocessingml')) {
    try {
      const result = await mammoth.extractRawText({ buffer });
      text = result.value || '';
    } catch (err) {
      throw new Error(`Failed to extract text from DOCX: ${err.message}`);
    }
  } else {
    // Plain text / markdown
    text = buffer.toString('utf-8');
  }

  // Normalize newlines and spaces
  const cleanText = text
    .replace(/\r\n/g, '\n')
    .replace(/\r/g, '\n')
    .replace(/[ \t]+/g, ' ')
    .trim();

  return {
    rawText: cleanText,
    fileHash: calculateFileHash(buffer),
    fileSize: buffer.length,
  };
}

/**
 * Extracts candidate basic contact info, education, and experience from text
 */
function parseResumeEntities(text) {
  const lines = text.split('\n').map((l) => l.trim()).filter(Boolean);

```

```
66 |    // 1. Email extraction
67 |    const emailRegex = /([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})/g;
68 |    const emailMatch = text.match(emailRegex);
69 |    const email = emailMatch ? emailMatch[0].toLowerCase() : '';
70 |
71 |    // 2. Phone extraction
72 |    const phoneRegex = /(?:(?:\+?1\s*(?:[.-]\s*)?)?(?:\(\s*([2-9]1[02-9]|[2-9][02-8]1|[2-9
   >  ][02-8][02-9])\s*\)|([2-9]1[02-9]|[2-9][02-8]1|[2-9][02-8][02-9]))\s*(?:[.-]\s*)?)?([2-9
   >  ]1[02-9]|[2-9][02-9]1|[2-9][02-9]{2})\s*(?:[.-]\s*)?([0-9]{4})(?:\s*(?:#|x\.?|ext\.?|ext
   >  ension)\s*(\d+))?|(\+?\d{1,3}[-.\s]?\(?\d{2,4}\)?[-.\s]?\d{3,4}[-.\s]?\d{3,4})/g;
73 |    const phoneMatch = text.match(phoneRegex);
74 |    const phone = phoneMatch ? phoneMatch[0].trim() : '';
75 |
76 |    // 3. Links (LinkedIn, GitHub, Portfolio)
77 |    const linkedinMatch = text.match(/(?:https?:\/\/)?(?:www\.)?linkedin\.com\/in\/([a-zA-
   >  Z0-9_-]+)/i);
78 |    const githubMatch = text.match(/(?:https?:\/\/)?(?:www\.)?github\.com\/([a-zA-Z0-9_-]+
   >  )/i);
79 |    const portfolioMatch = text.match(/(?:https?:\/\/)?([a-zA-Z0-9_-]+\.(?:dev|me|io|com|o
   >  rg))(?:\/[^\s]*)?/i);
80 |
81 |    const links = {
82 |      linkedin: linkedinMatch ? linkedinMatch[0] : '',
83 |      github: githubMatch ? githubMatch[0] : '',
84 |      portfolio: portfolioMatch && !portfolioMatch[0].includes('linkedin') && !portfolioMa
   >  tch[0].includes('github') ? portfolioMatch[0] : '',
85 |    };
86 |
87 |    // 4. Candidate Name: Usually first 1-3 lines before email / phone
88 |    let name = 'Candidate';
89 |    for (let i = 0; i < Math.min(lines.length, 5); i++) {
90 |      const line = lines[i];
91 |      if (
92 |        line.length > 2 &&
93 |        line.length < 40 &&
94 |        !line.includes('@') &&
95 |        !line.match(/\d{5,}/) &&
96 |        !line.toLowerCase().includes('resume') &&
97 |        !line.toLowerCase().includes('curriculum') &&
98 |        !line.toLowerCase().includes('http')
99 |      ) {
100 |        name = line.replace(/[^a-zA-Z\s.'-]/g, '').trim();
101 |        if (name.split(/\s+/).length >= 1 && name.length > 2) {
102 |          break;
103 |        }
104 |      }
105 |    }
106 |
107 |    // 5. Total Years of Experience estimation
108 |    let totalExperienceYears = 0;
109 |    // Look for "X+ years of experience"
110 |    const expMatch = text.match(/(\d+)\+?\s*years?(?:\s+of)?\s+(?:experience|working)/i);
111 |    if (expMatch && parseInt(expMatch[1], 10) < 35) {
112 |      totalExperienceYears = parseInt(expMatch[1], 10);
113 |    } else {
114 |      // Estimate by counting year intervals (e.g. 2018 - 2023, 2021 - Present)
115 |      const yearRangeRegex = /(20\d{2}|19\d{2})\s*(?:-|â€“|â€”|to)\s*(20\d{2}|present|current)
   >  /gi;
116 |      let match;
117 |      let minYear = 2050;
118 |      let maxYear = 1970;
119 |      const currentYear = new Date().getFullYear();
120 |
121 |      while ((match = yearRangeRegex.exec(text)) !== null) {
122 |        const start = parseInt(match[1], 10);
123 |        const end = match[2].toLowerCase().includes('present') || match[2].toLowerCase().i
   >  ncludes('current')
124 |          ? currentYear
125 |          : parseInt(match[2], 10);
126 |
127 |        if (start >= 1980 && start <= currentYear && end >= start) {
128 |          minYear = Math.min(minYear, start);
```

```
129 |           maxYear = Math.max(maxYear, end);
130 |         }
131 |       }
132 |
133 |       if (minYear <= currentYear && maxYear >= minYear) {
134 |         totalExperienceYears = Math.min(maxYear - minYear, 30);
135 |       }
136 |       if (totalExperienceYears === 0) {
137 |         totalExperienceYears = 3; // sensible default
138 |       }
139 |     }
140 |
141 |     // 6. Education detection
142 |     const educationTimeline = [];
143 |     const eduKeywords = [
144 |       { degree: "Ph.D.", pattern: /(?:ph\.?d\.?|doctor\s+of\s+philosophy)/i },
145 |       { degree: "Master's Degree (M.S. / M.Tech / M.Sc)", pattern: /(?:master|m\.s\.|m\.te
    > ch|m\.sc|msc|mba)/i },
146 |       { degree: "Bachelor's Degree (B.S. / B.Tech / B.E.)", pattern: /(?:bachelor|b\.s\.|b
    > \.tech|b\.e\.|bsc|b\.a\.)/i },
147 |       { degree: "Associate Degree / Diploma", pattern: /(?:associate|diploma)/i },
148 |     ];
149 |
150 |     for (const item of eduKeywords) {
151 |       if (item.pattern.test(text)) {
152 |         educationTimeline.push({
153 |           degree: item.degree,
154 |           institution: 'University / Institute',
155 |           field: 'Computer Science / Engineering / Design',
156 |           graduationYear: 'Graduated',
157 |         });
158 |         break; // Record primary degree level
159 |       }
160 |     }
161 |
162 |     if (educationTimeline.length === 0) {
163 |       educationTimeline.push({
164 |         degree: "Bachelor's Degree (Equivalent)",
165 |         institution: 'Recognized University',
166 |         field: 'Relevant Domain',
167 |         graduationYear: 'Graduated',
168 |       });
169 |     }
170 |
171 |     // 7. Work timeline heuristic segmentation
172 |     const experienceTimeline = extractTimelineSections(lines, text);
173 |
174 |     // 8. Quality Analyzer
175 |     const qualityIssues = analyzeResumeQuality(text, {
176 |       name,
177 |       email,
178 |       phone,
179 |       totalExperienceYears,
180 |       educationTimeline,
181 |       experienceTimeline,
182 |     });
183 |
184 |     return {
185 |       name,
186 |       email,
187 |       phone,
188 |       links,
189 |       totalExperienceYears,
190 |       educationTimeline,
191 |       experienceTimeline,
192 |       qualityIssues,
193 |     };
194 |   }
195 |
196 |   /**
197 |    * Extracts approximate work experience blocks
198 |    */
```

```
199 |  function extractTimelineSections(lines, fullText) {
200 |    const timeline = [];
201 |    const datePattern = /(?:(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*\.?\s+)
    >  ?(20\d{2}|19\d{2})\s*(?:-|â€“|â€”|to)\s*(?:(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)
    >  [a-z]*\.?\s+)?(20\d{2}|present|current)/i;
202 |
203 |    let currentEntry = null;
204 |
205 |    for (let i = 0; i < lines.length; i++) {
206 |      const line = lines[i];
207 |      const match = line.match(datePattern);
208 |
209 |      if (match) {
210 |        if (currentEntry) {
211 |          timeline.push(currentEntry);
212 |        }
213 |
214 |        // Try to get role and company from nearby lines
215 |        const prevLine = i > 0 ? lines[i - 1] : '';
216 |        const nextLine = i + 1 < lines.length ? lines[i + 1] : '';
217 |
218 |        let role = prevLine.length < 60 ? prevLine : 'Software Engineer';
219 |        let company = 'Enterprise Technology';
220 |
221 |        if (role.includes(' at ')) {
222 |          const parts = role.split(' at ');
223 |          role = parts[0].trim();
224 |          company = parts[1].trim();
225 |        } else if (role.includes(' - ')) {
226 |          const parts = role.split(' - ');
227 |          role = parts[0].trim();
228 |          company = parts[1].trim();
229 |        }
230 |
231 |        currentEntry = {
232 |          role: role.length > 5 ? role : 'Senior Specialist',
233 |          company: company,
234 |          startDate: match[0].split(/-|â€“|â€”|to/i)[0].trim(),
235 |          endDate: match[0].split(/-|â€“|â€”|to/i)[1].trim(),
236 |          isCurrent: match[0].toLowerCase().includes('present') || match[0].toLowerCase().
    >  includes('current'),
237 |          description: nextLine.length > 30 ? nextLine : 'Designed and developed high-impa
    >  ct features and services.',
238 |          technologies: [],
239 |        };
240 |      }
241 |    }
242 |
243 |    if (currentEntry) {
244 |      timeline.push(currentEntry);
245 |    }
246 |
247 |    // Fallback if no dates matched nicely
248 |    if (timeline.length === 0) {
249 |      timeline.push({
250 |        role: 'Software Engineer',
251 |        company: 'Technology Solutions',
252 |        startDate: '2020',
253 |        endDate: 'Present',
254 |        isCurrent: true,
255 |        description: 'Full stack development, cloud microservices, and system architecture
    >  .',
256 |        technologies: [],
257 |      });
258 |    }
259 |
260 |    return timeline;
261 |  }
262 |
263 |  /**
264 |   * Resume Quality Analyzer (Section 4, Feature 1)
265 |   */
```

```
266 |  function analyzeResumeQuality(text, extracted) {
267 |    const issues = [];
268 |
269 |    // Check 1: Missing contact info
270 |    if (!extracted.email) {
271 |      issues.push({
272 |        type: 'missing_section',
273 |        severity: 'medium',
274 |        description: 'No valid email address was detected on the resume.',
275 |        suggestion: 'Ensure email address is prominently placed in the header section.',
276 |      });
277 |    }
278 |
279 |    if (!extracted.phone) {
280 |      issues.push({
281 |        type: 'missing_section',
282 |        severity: 'low',
283 |        description: 'No phone number was detected.',
284 |        suggestion: 'Include direct phone contact for recruiter outreach.',
285 |      });
286 |    }
287 |
288 |    // Check 2: Content depth
289 |    if (text.length < 500) {
290 |      issues.push({
291 |        type: 'formatting_warning',
292 |        severity: 'high',
293 |        description: 'Resume is unusually brief (under 100 words), limiting screening accu
    >  racy.',
294 |        suggestion: 'Provide detailed descriptions of responsibilities, technical stacks,
    >  and measurable outcomes.',
295 |      });
296 |    }
297 |
298 |    // Check 3: Check for vague skill descriptions
299 |    if (!text.toLowerCase().includes('skills') && !text.toLowerCase().includes('technical
    >  competencies')) {
300 |      issues.push({
301 |        type: 'unclear_skill_description',
302 |        severity: 'low',
303 |        description: 'No dedicated "Skills" or "Core Competencies" section header detected
    >  .',
304 |        suggestion: 'Group key technical and leadership competencies under a dedicated hea
    >  der.',
305 |      });
306 |    }
307 |
308 |    return issues;
309 |  }
310 |
311 |  module.exports = {
312 |    extractRawText,
313 |    parseResumeEntities,
314 |    calculateFileHash,
315 |  };
316 |
```